Listing 14.3 Reading from and writing to Kafka

```
object ConsumerProducerAtLeastOnceConsumerProducerAtleastOnce {

  implicit val system = ActorSystem(Behaviors.empty, "producerOne")

  implicit val ec = ExecutionContext.Implicits.global

  def main(args: Array[String]) = {

    val bootstrapServers = "127.0.0.1:9092"

    val consumerConfig =
      system.settings.config.getConfig("akka.kafka.consumer")

    val consumerSettings: = ConsumerSettings(
         consumerConfig,
         new StringDeserializer(),
         new StringDeserializer())                                    ❶
        .withBootstrapServers(bootstrapServers)
        .withGroupId("group01")                                       ❷
        .withProperty(
          ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,                    ❸
          "earliest")

    val producerConfig =
      system.settings.config.getConfig("akka.kafka.producer")

    val producerSettings = ProducerSettings(
      producerConfig, new StringSerializer(),new StringSerializer())  ❹
      .withBootstrapServers(bootstrapServers)

    val drainingControl: Consumer.DrainingControl[_] = Consumer
      .committableSource(consumerSettings, Subscriptions.topics("test3"))  ❺
      .map { msg: CommittableMessage[String, String] =>
        ProducerMessage.single(
          new ProducerRecord[String, String](
            "test4",msg.record.key, msg.record.value),                ❻
            msg.committableOffset)                                    ❼
      }
      .toMat(Producer.committableSink(producerSettings))(            ❽
        Consumer.DrainingControl.apply)
      .run()

    StdIn.readLine("Consumer started \n Press Enter to stop")
    val future = drainingControl.drainAndShutdown
    future.onComplete(_ => system.terminate)
```